

Sui의 확장성 개선 방법

이준서



목차

- **Sui System Architecture, Sui Lutris**
 - Hybrid Consensus Architecture
- **Narwhal Mempool**
 - Proof of Availability

Smart contract platform, Sui

- 독자적인 Layer1 설계를 통한 확장성 개선
- Why not Layer2 ?
 - Layer2 Solution의 확장성 개선 한계
 - Layer2 생태계에서는 독립적이고 고립되어 있는 상태(State)를 가지고 있음
 - Layer2와 Layer1과의 즉각적인 동기화 불가능
 - 스마트 컨트랙트 플랫폼의 사용성 측면에서 일정 부분 희생할 수 밖에 없음
- “Build without boundaries”
 - “스마트 컨트랙트 플랫폼의 유틸리티는 플랫폼 위에서 상태(State)를 공유하며 작동하는 프로그램들이 서로 아토믹하게 접근하며 흥미로운 상호작용을 일으킬 수 있는 경우가 많아질 수록 높아진다고 생각한다”
 - by Sam Blackshear(CTO of Mysten Labs)

Sui System Architecture, Sui Lutris



기존 블록체인의 합의 알고리즘의 한계

- 모든 트랜잭션에 대한 Global Ordering
 - 높은 지연시간과 낮은 처리량
 - Ordering이 불필요한 트랜잭션도 정렬

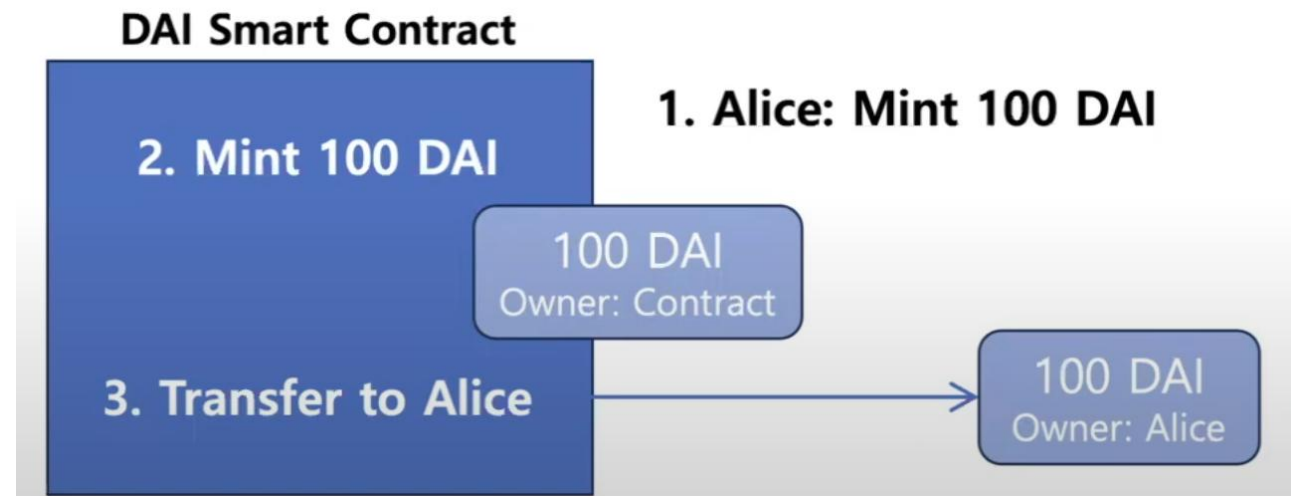
Sui Lutris

- Sui의 시스템 아키텍처
- 핵심 아이디어
 - Ordering 불필요한 트랜잭션에 대해서는 합의 X
 - Reliable Broadcast 기반 Consensus-less와 Ordering을 위한 Consensus 둘 다 사용
 - Hybrid Consensus Architecture

Ordering 불필요한 트랜잭션 판별

➤ Sui의 Smart Contract 언어 Move의 Object-based Data Model 사용

- **Single-owned Object**
 - 특정 계정 하나가 유일하게 소유(Assets)
 - Ordering 불필요
- **Shared Object**
 - 여러 트랜잭션이나 주체가 동시에 접근, 수정 가능(Smart Contract)
 - Ordering 필요
- **Combining UTXO model & Account based model**



<Object-based Data Model 예시>

Move 언어로 해결하려는 문제

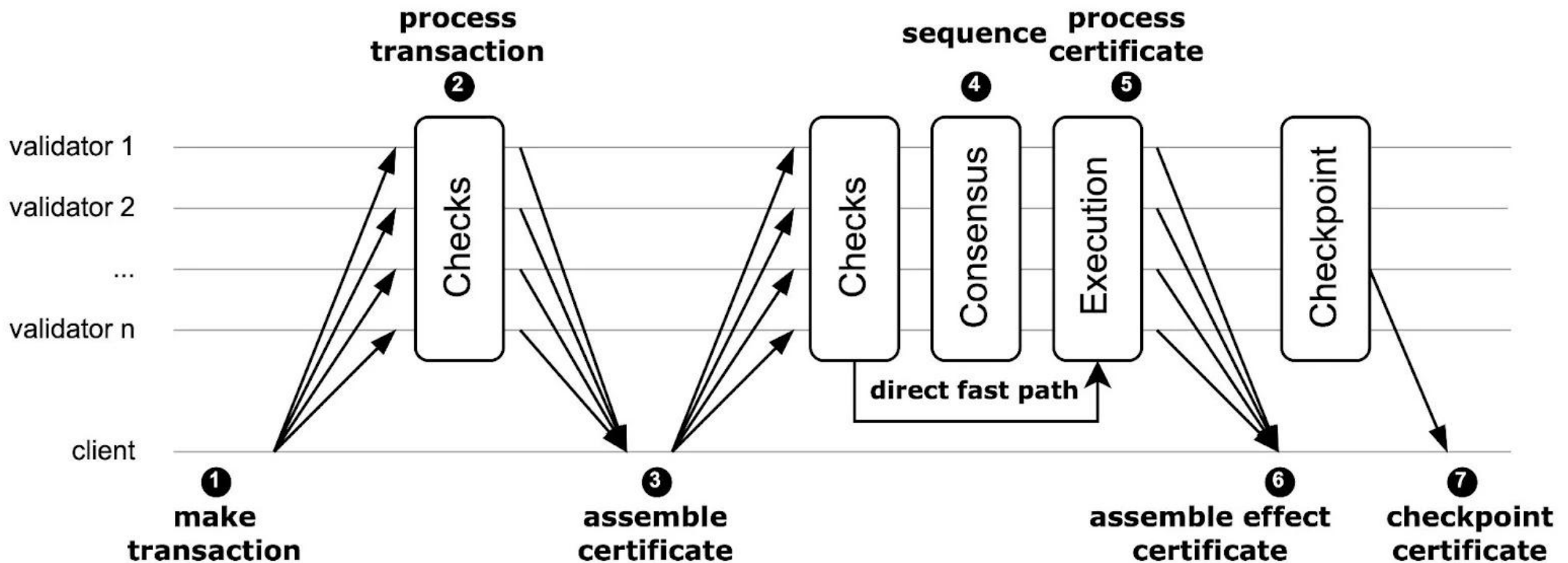
- 트랜잭션 Ordering의 필요성을 정적으로 구분 가능해야 함
- 블록체인은 자산이 존재하는 네트워크로, 자유롭게 자산을 생성하고, 효율적으로 자산을 이동할 수 있어야 한다.
 - Even Cheng(Sui 공동 창립자 중 한 명)
 - 소유권에 대한 증빙이 블록체인에 저장되어야 한다고 주장

Sui Lutris

- Sui의 시스템 아키텍처
- 핵심 아이디어
 - **Reliable Broadcast 기반 Consensus-less**
 - Single-Owned Object
 - **Ordering을 위한 Consensus**
 - Shared Object

Lutris의 트랜잭션 Life cycle

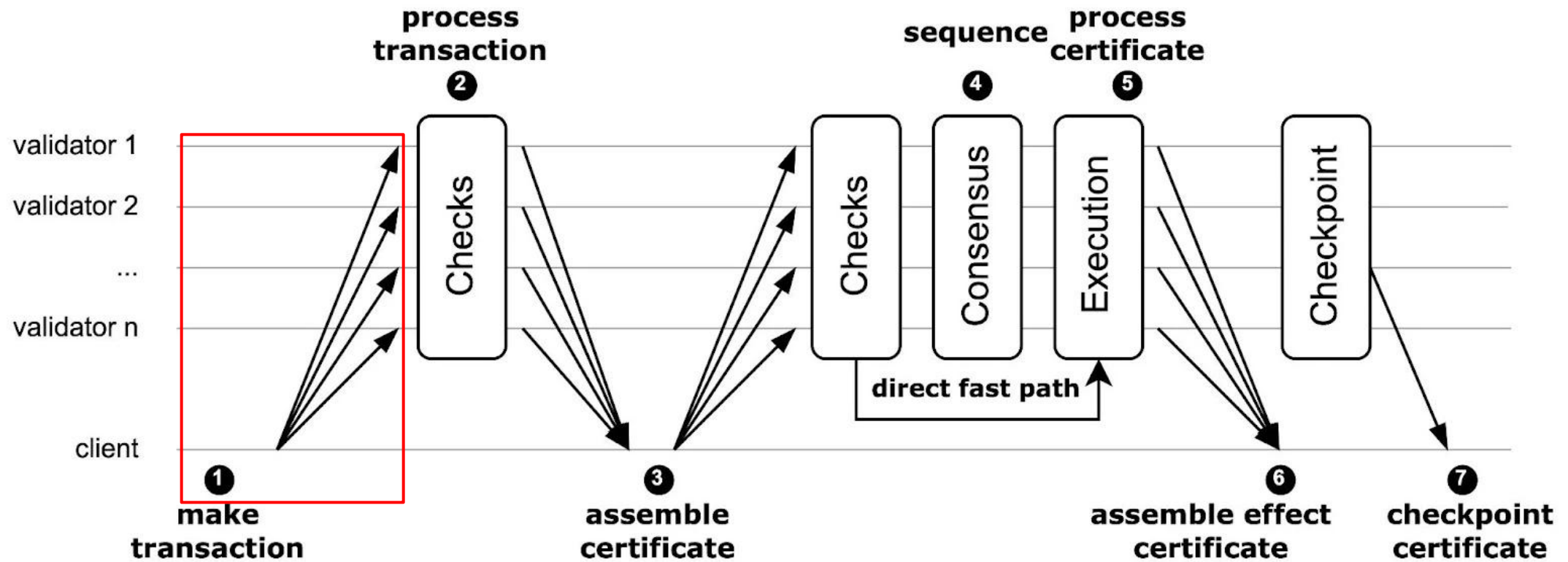
- 현재 Sui Mainnet 구현 상 Client가 아닌 Full node가 Certificate를 모음



<Transaction Lifecycle>

Lutris의 트랜잭션 Life cycle

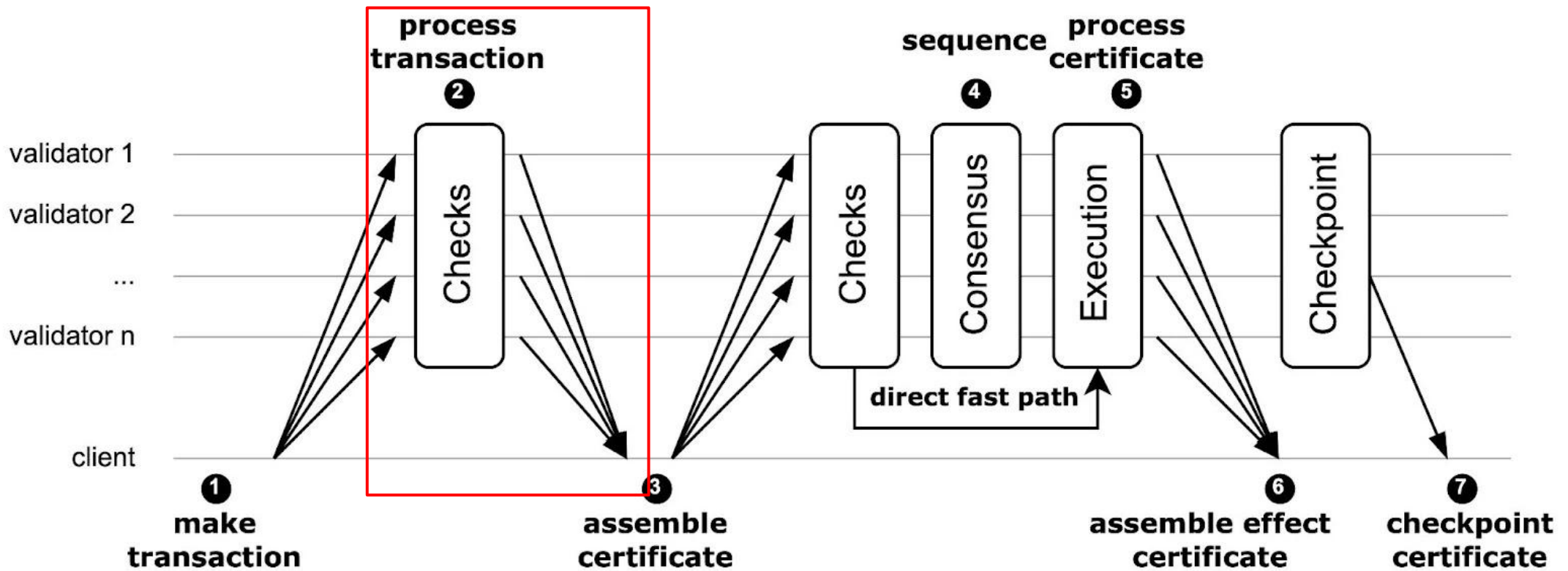
- Client는 검증자들에게 트랜잭션 broadcast



<Transaction Lifecycle>

Lutris의 트랜잭션 Life cycle

- 검증자들은 트랜잭션에 대해 검증하고 Sign하고 다시 보냄
 - Prevalidation (No execution)
 - Owned Object에 대해서는 검증자가 Locally lock (Double spending 방지)

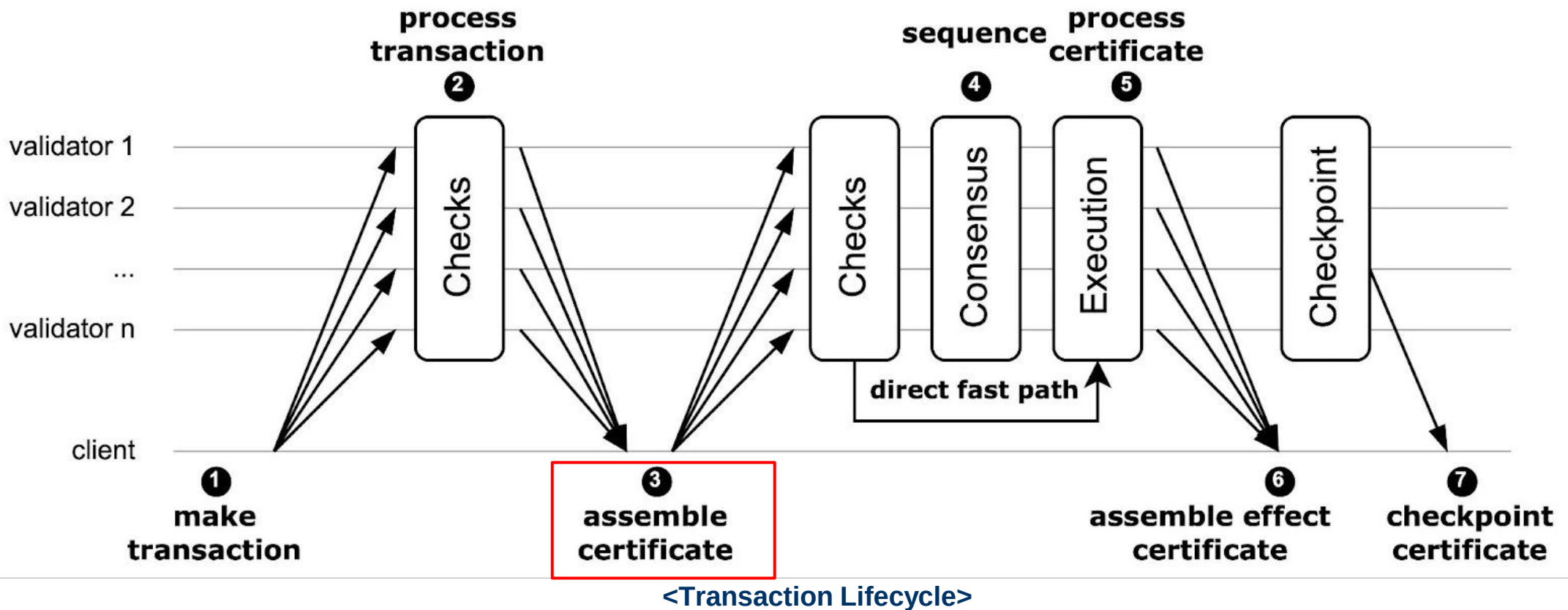


<Transaction Lifecycle>

Lutris의 트랜잭션 Life cycle

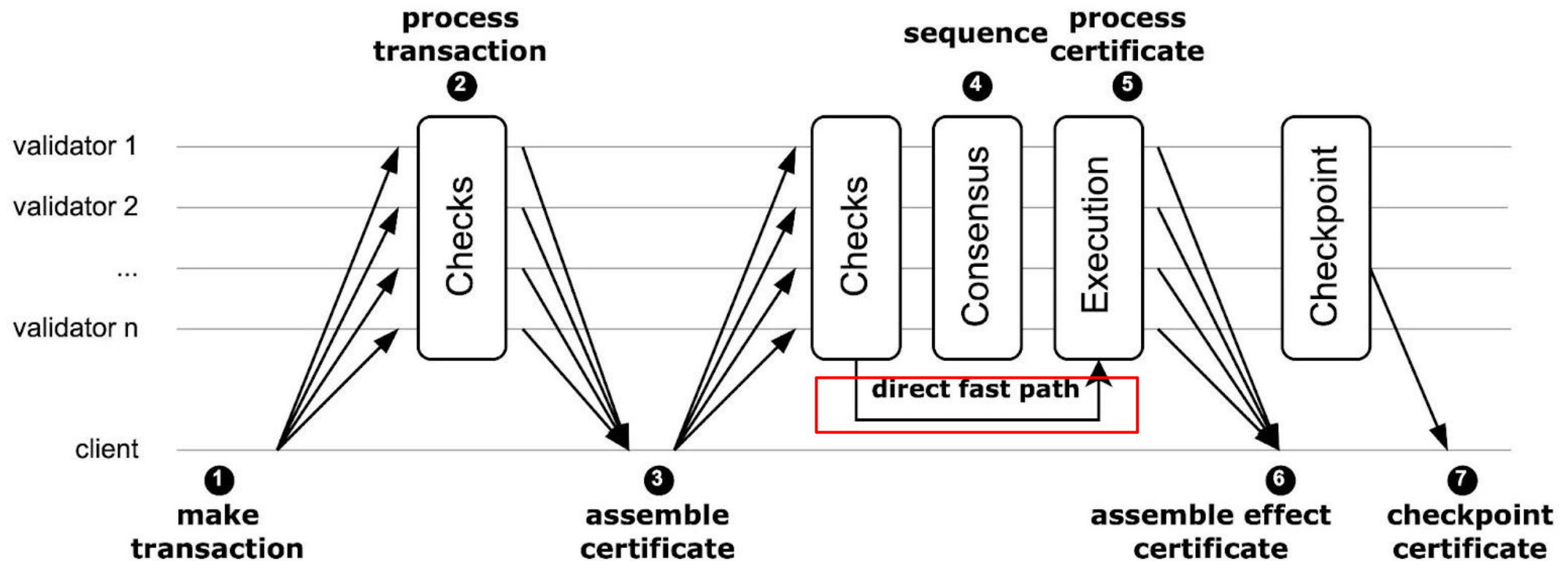
➤ N-f Validators에게 응답 모아서 Transaction Certificate 모음

- Certificate을 다시 Validator에게 Broadcast
 - Transaction Finality(트랜잭션 실행에 대한 Finality)



Lutris의 트랜잭션 Life cycle

- ▶ 트랜잭션에 Owned Object만 있다면 Consensus 없이 실행
 - Ordering 필요 없으니 합의 X



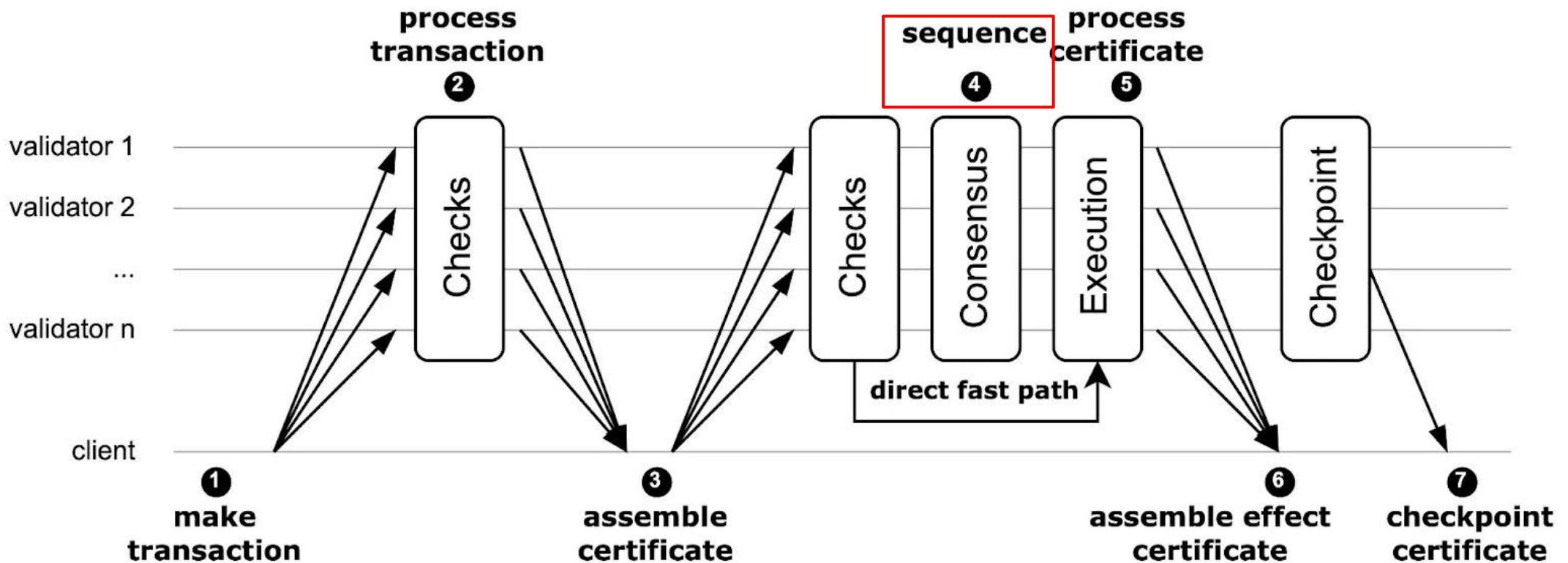
<Transaction Lifecycle>

Lutris의 트랜잭션 Life cycle

➤ 모든 Certificate를 Consensus Protocol에 전송

▪ Owned Object first

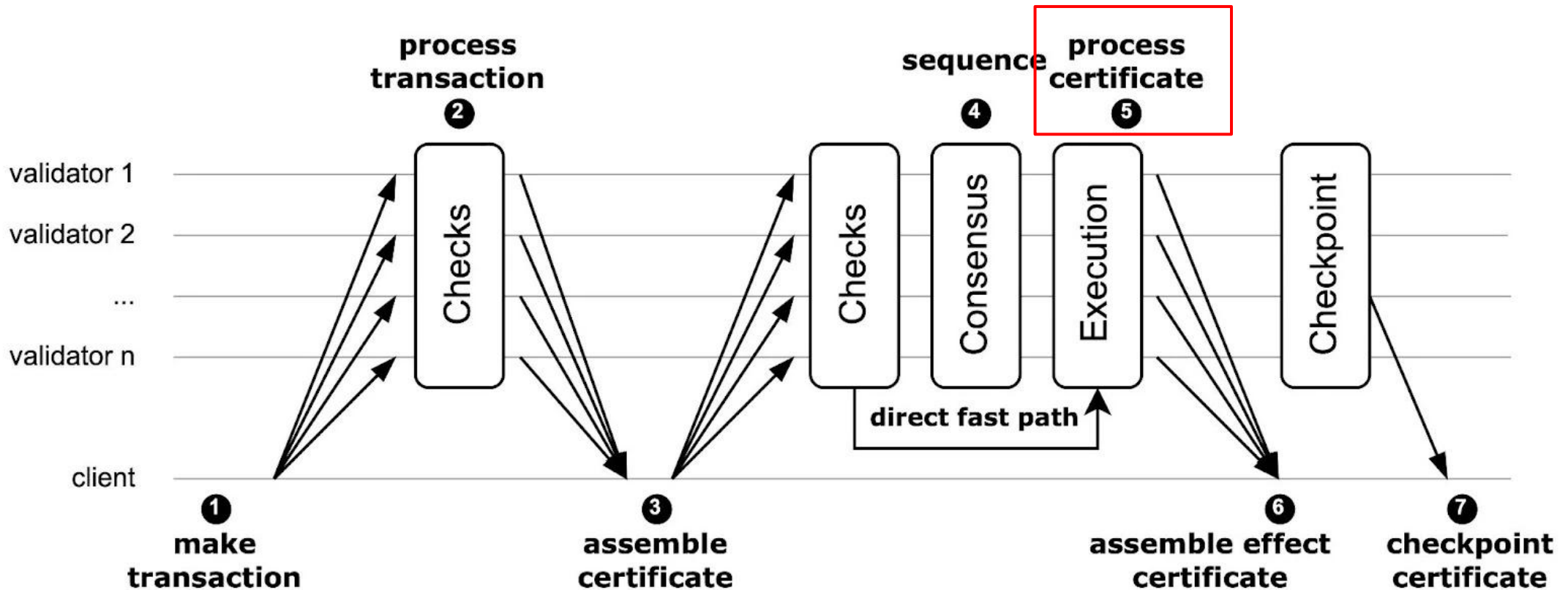
- Owned Object끼리의 순서는 상관 없음



<Transaction Lifecycle>

Lutris의 트랜잭션 Life cycle

- Validators가 Shared Object에 대해서 트랜잭션 실행
 - Settlement Finality (실행 결과에 대한 Finality)
 - 특이하게도 Finality 개념을 두가지로 가지고 감(Transaction Finality, Settlement Finality)

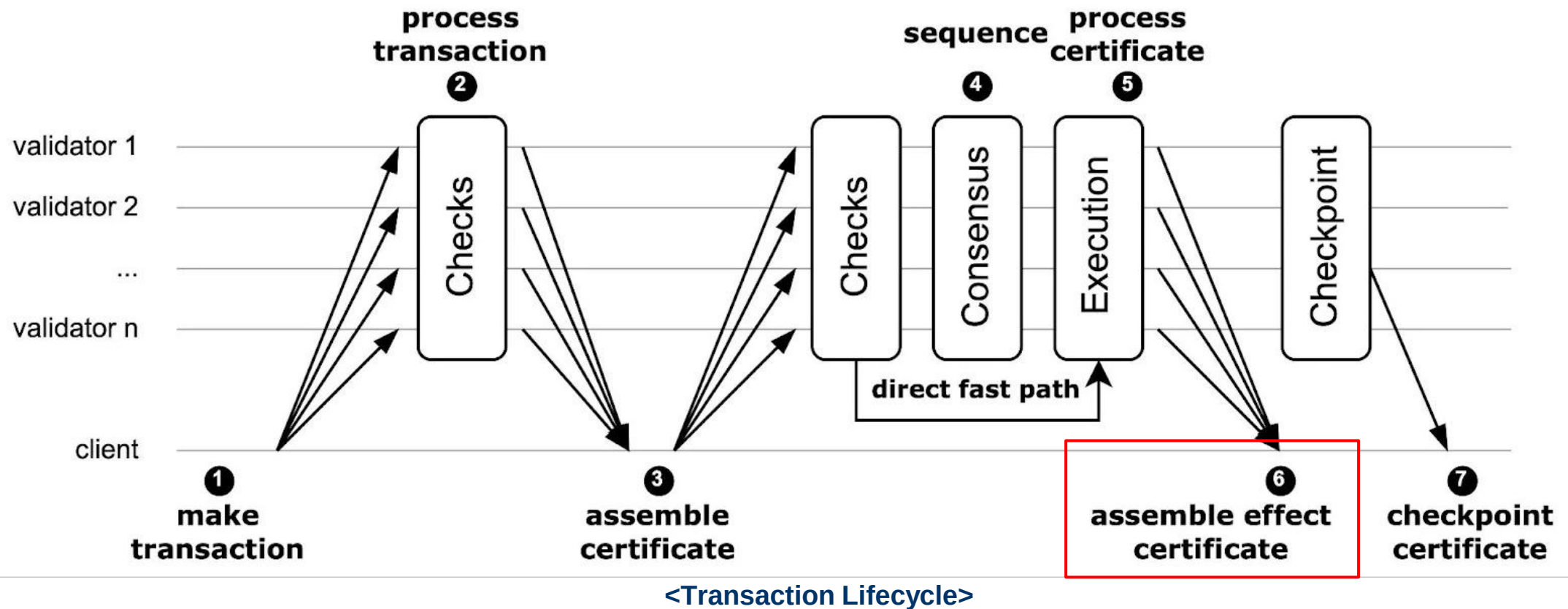


<Transaction Lifecycle>

Lutris의 트랜잭션 Life cycle

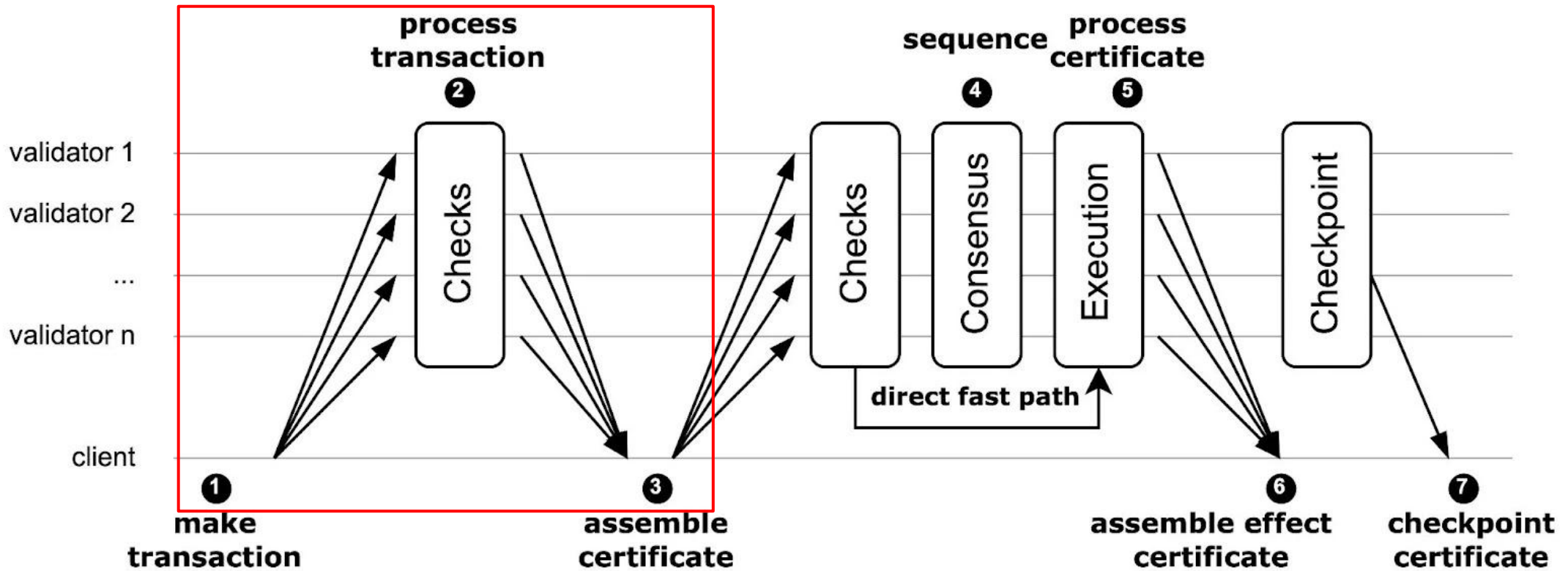
➤ 2f+1 실행 결과 모아서 Effect Certificate 생성

- 트랜잭션의 최종 확정 의미
 - Checkpoint찍을 때 사용



Lutris 구조에 대한 의문점

- 다른 블록체인과 다르게 합의를 해야 하는 트랜잭션에 대해서도 Reliable Broadcast
 - Narwhal Mempool에서의 과정
 - 한번 검증을 하는데 합의할 때 또 검증을 한다면 비효율적?



<Transaction Lifecycle>

Narwhal DAG-based Mempool



Mempool의 정의

- 블록체인에서 아직 블록에 포함되지 않은 트랜잭션들이 일시적으로 저장되는 공간
 - 일반적으로 Queue의 형태

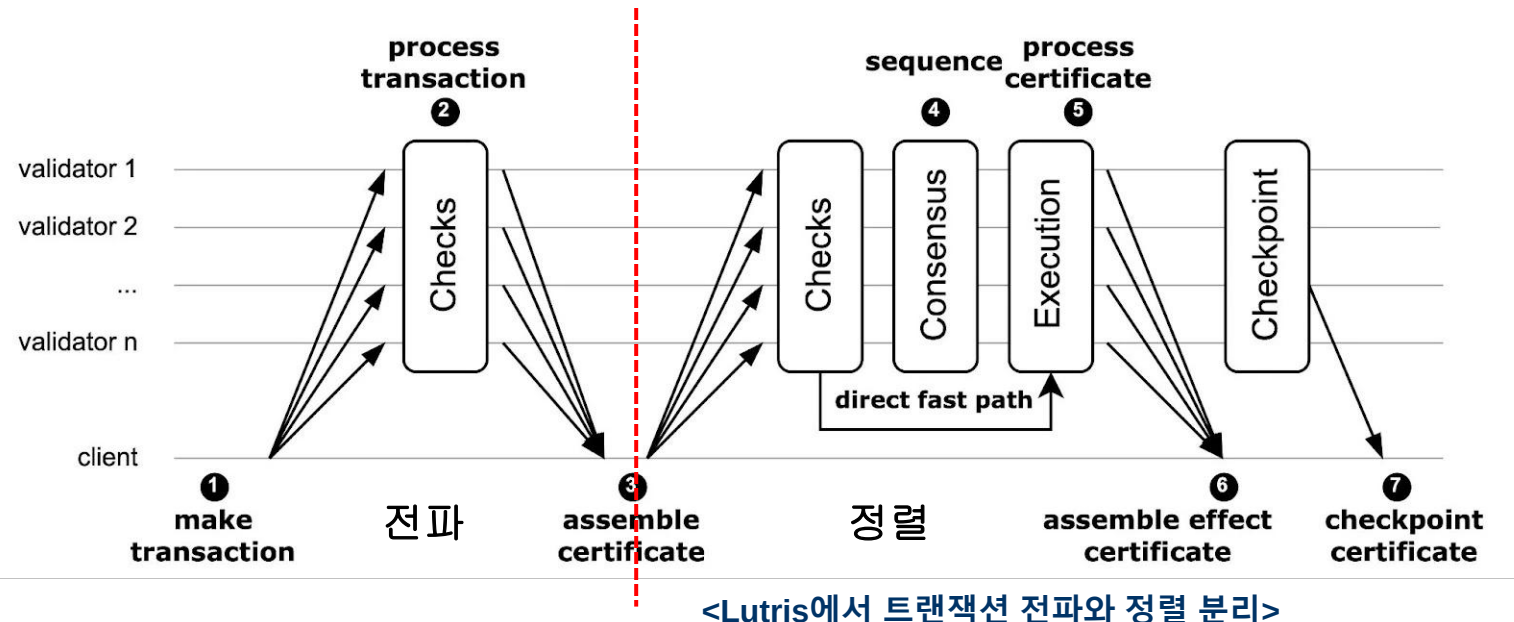
Narwhal에서의 Mempool 개념 확장

- Data Availability 보장을 위한 Layer
 - 각 노드가 받은 트랜잭션을 네트워크 상에서 공유, 저장하고 데이터 가용성 보장
 - 결국엔 모든 정직한 노드가 유효한 데이터를 보게 됨
- 받은 트랜잭션들을 DAG형태의 자료구조로 만들어 놓음

Lutris에서의 트랜잭션 전파와 정렬의 분리

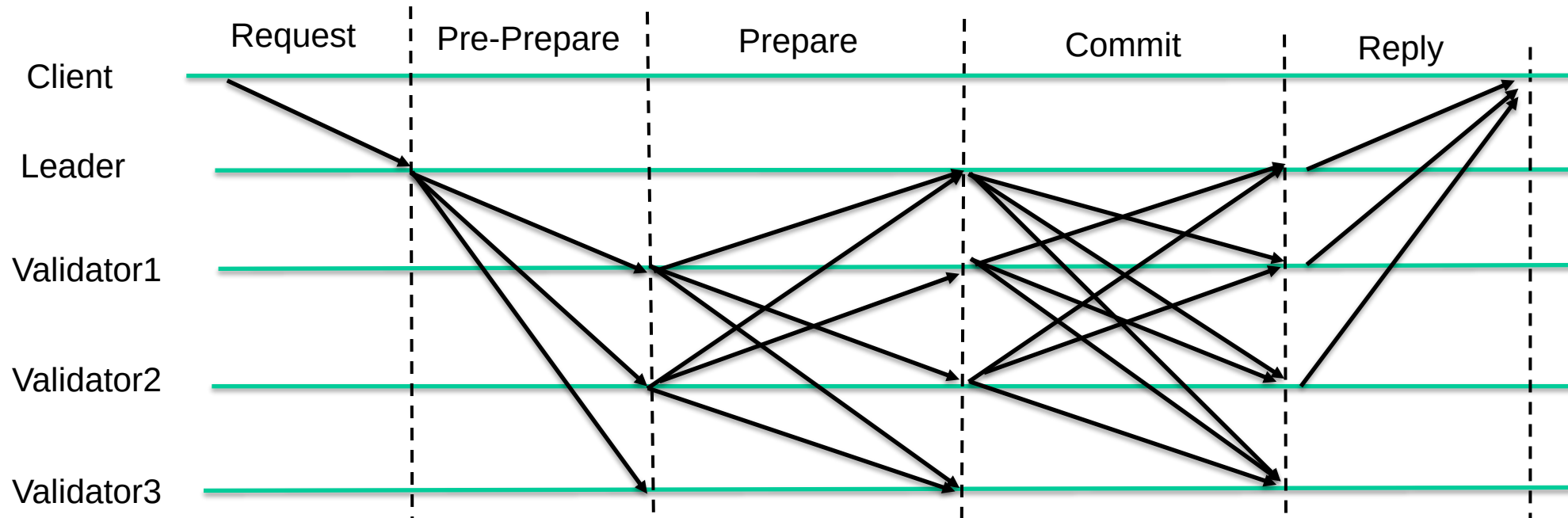
- 트랜잭션 전파 Layer에서는 데이터 가용성 보장
 - 모든 노드가 동일하고 유효한 트랜잭션을 봐야함
 - Narwhal Mempool

- 트랜잭션 정렬 Layer에서는 Total Ordering 확정
 - 모든 노드가 동일한 순서로 트랜잭션을 실행해야함
 - Tusk, Bullshark, Hotstuff, ...



PBFT 합의 과정

- PBFT는 리더가 Client에게 모든 트랜잭션을 받아서 처리
 - 리더만 Mempool 유지하면 됨
 - Queue 형태의 Mempool
- 하나의 프로토콜 안에서 트랜잭션 전파와 정렬 한 번에 진행



<PBFT 합의 프로토콜 진행 방식>

Narwhal 통한 확장성 문제 해결

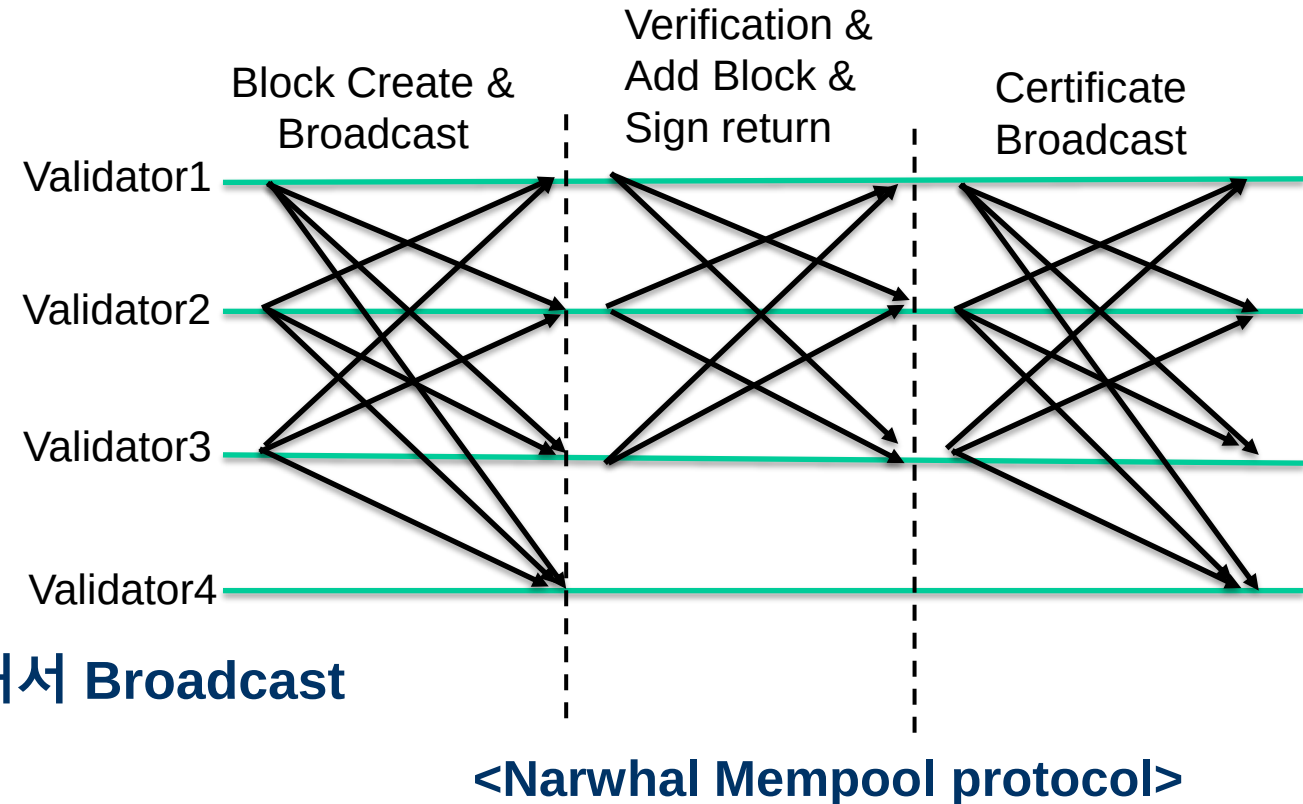
➤ Proof of Availability

- 트랜잭션 전파와 정렬의 분리를 통한 병렬 처리 가능
- 합의할 때는 미리 전파한 트랜잭션에 대한 인증서로 합의 진행
 - 네트워크 트래픽 절약
- 리더만 Client에게 트랜잭션을 받지 않고 모든 노드가 트랜잭션을 받을 수 있음
 - 리더 병목현상 완화

➤ DAG형태로 트랜잭션들이 자료구조화 되어있어서 합의 횟수 감소

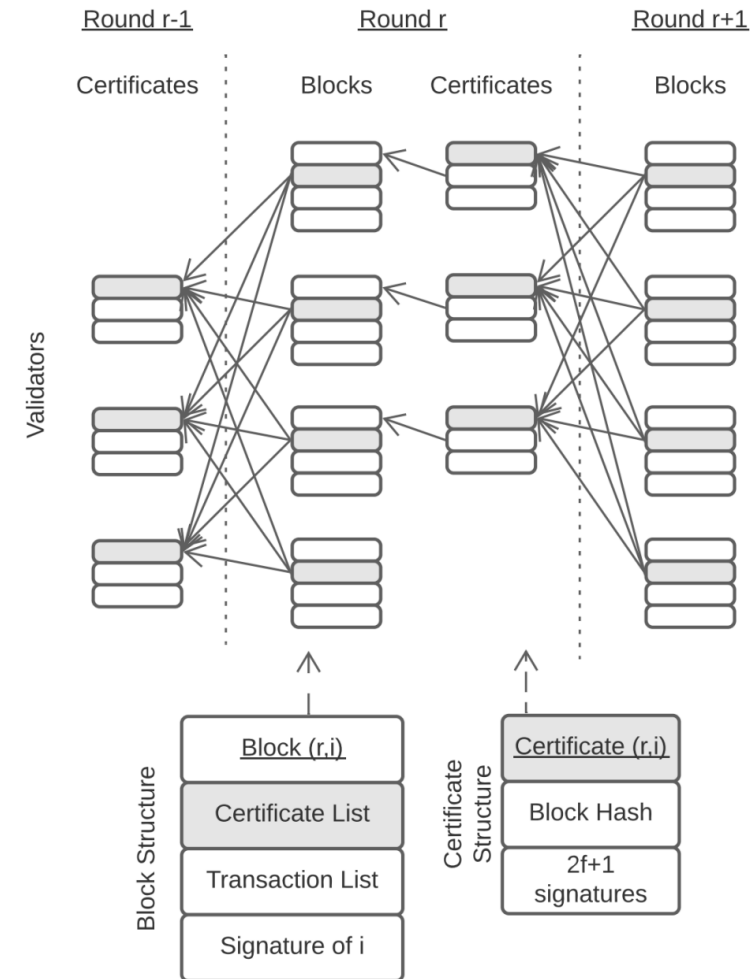
Narwhal Mempool Protocol

- 각 검증자는 로컬 라운드 r 유지
 - 라운드마다 블록 하나씩 생성
- Block 생성해서 Broadcast
 - Block에는 다음 정보 포함됨
 - 이전 라운드 $2f+1$ 인증서 List
 - 트랜잭션 List
 - 블록 생성자의 Signature
- 메시지 받은 검증자들은 블록 검증
 - 유효한 블록이면 Signature로 응답
- $2f+1$ 개의 Signature받으면 인증서 생성해서 Broadcast
- $2f+1$ 개의 인증서 모으면 다음 라운드 넘어갈 수 있음



DAG형태의 Mempool

- 각 트랜잭션 블록이 DAG형태로 저장됨
 - Vertex: 트랜잭션 batch
 - Edge: 이전 라운드의 batch로의 Reference
 - 모든 Certificate으로의 reference



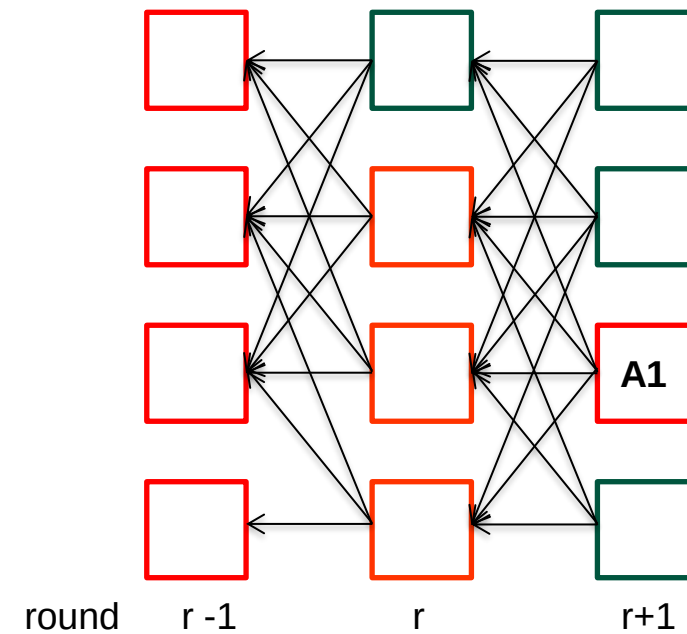
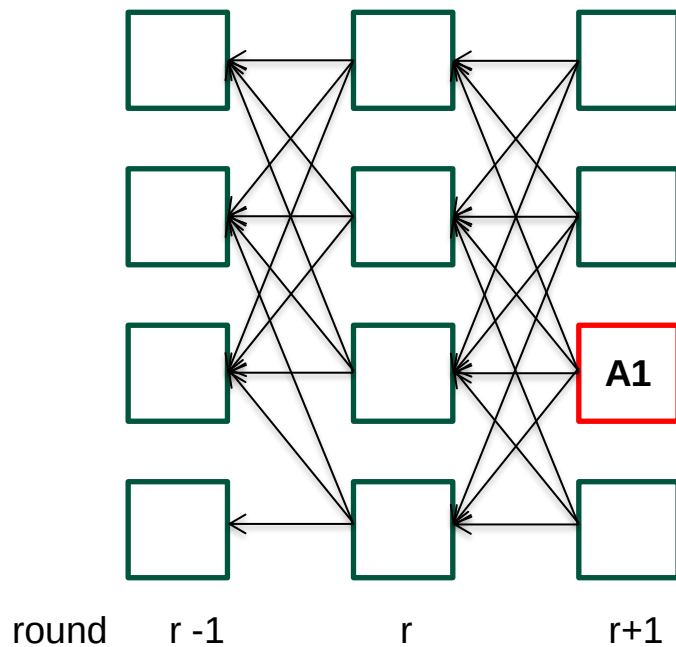
<Narwhal DAG-based Mempool>

Narwhal과 합의 프로토콜 결합

- 트랜잭션 전파와 정렬의 분리
 - 전파는 Narwhal Mempool을 통해서
 - 정렬은 여러 합의 알고리즘으로 가능
- Narwhal으로 DAG-based Mempool을 만들고 합의는 리더가 있는 합의 알고리즘 사용 가능
 - 리더가 트랜잭션 블록을 제안하고 검증자들이 검증하는 방식
- 리더가 트랜잭션 블록 직접 제안하는 대신 Narwhal에서 생성된 가용성 증명서 제안
 - 해당 블록이 존재하고 검증되었고 검색 가능함을 나타내는 증명서
 - DAG형태로 연결되어 있어서 이전 라운드의 블록도 같이 Commit 가능
 - DAG형태에서 Child 모두 Commit 대상
 - ✓ Child가 DAG내에서 QC 확보하였다면 Commit
 - ✓ QC확보하지 못한 상태로 시간 지나면 GC 대상

DAG기반 Mempool & 리더 기반 합의알고리즘 결합

- DAG형태의 Mempool에서 round번호가 높은 Vertex의 블록에 대한 합의 진행
- A1에 대한 합의 완료하면 DAG 내의 edge로 연결되어있는 Child Commit
 - Child에 대한 Ordering은 리더가 DFS 방식으로 순서 정함
 - Narwhal-HS에서는 DFS 방식으로 순서를 정했지만 Some Deterministic Order로 Commit

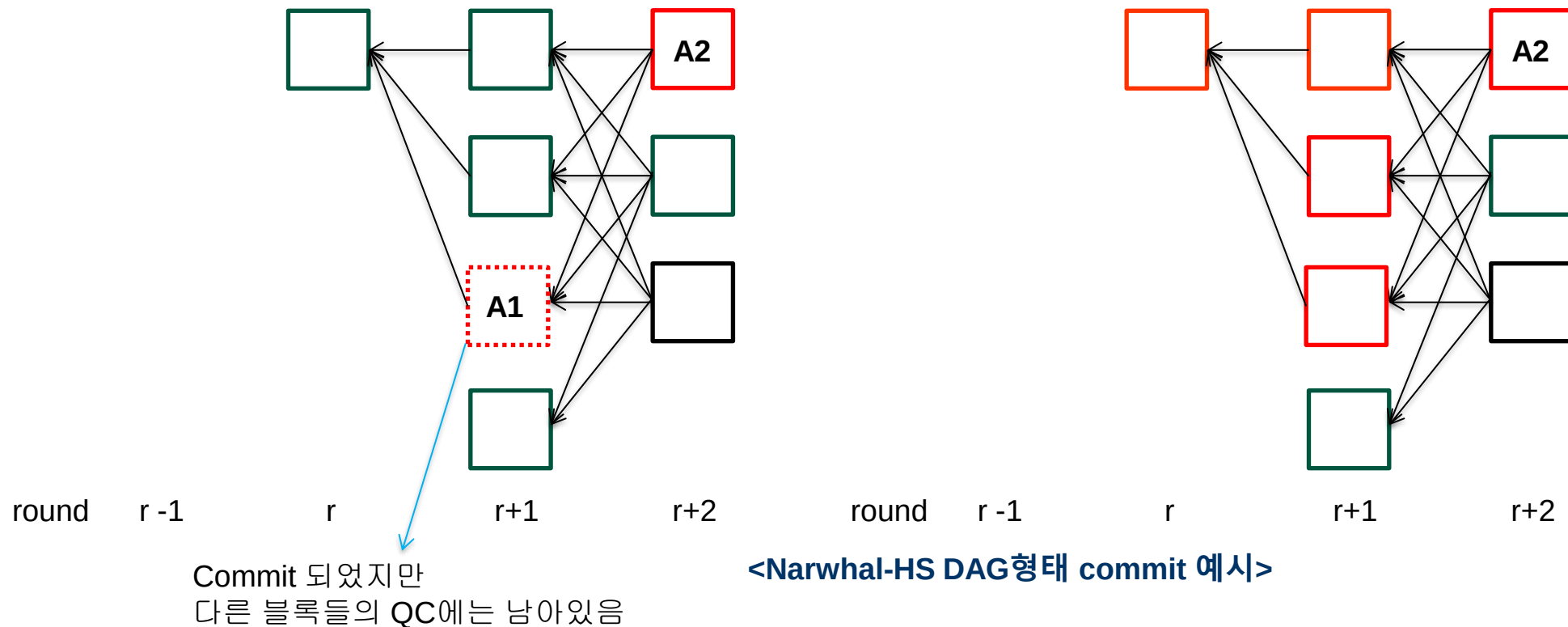


 Commit 대상

<Narwhal-HS DAG형태 commit 예시>

다음 라운드의 Commit 대상 블록

- 구현체에서는 이중 Commit을 막기 위해 DAG에서 Commit 되면 삭제시켜줌
- 다음 Leader의 Propose에서 남은 DAG 대상으로 합의 진행

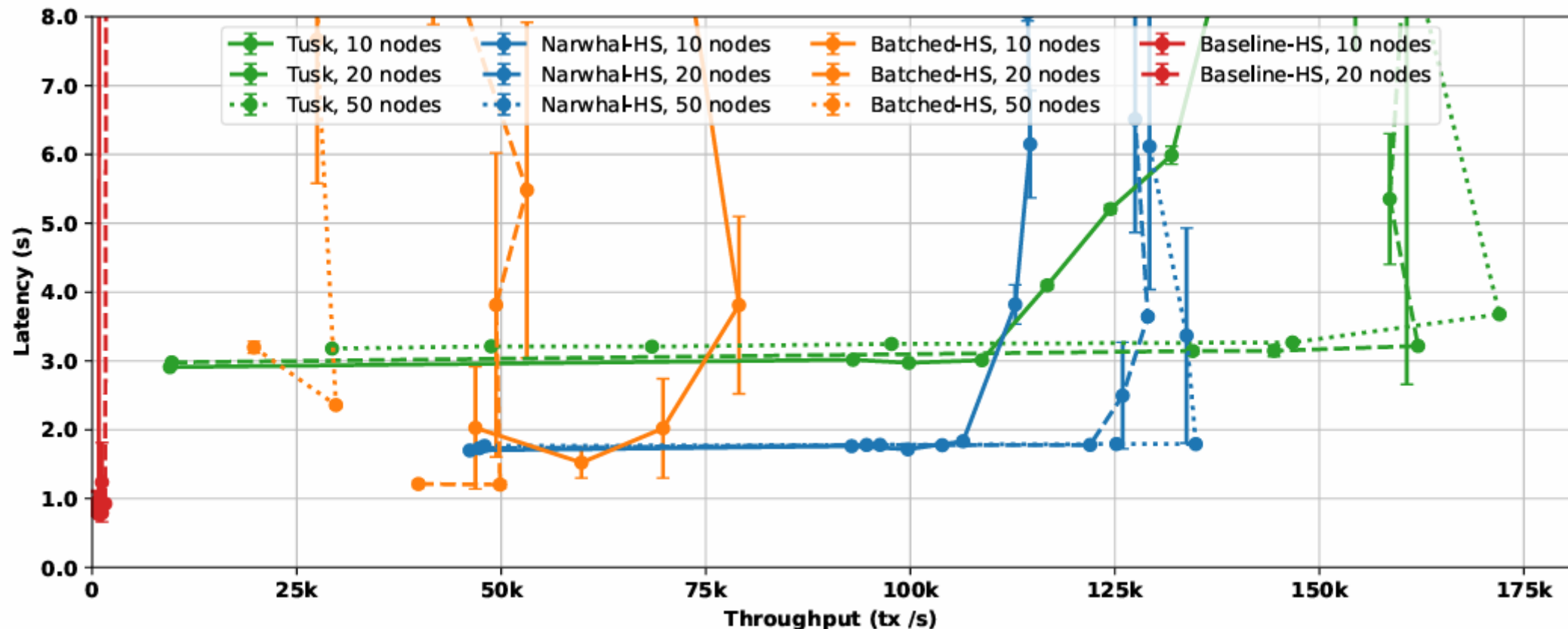


Narwhal과 결합하는 리더 기반 합의 알고리즘의 역할

- 리더 기반 합의알고리즘에서 합의하는 블록을 Tree의 root로 생각
 - DAG로 연결되어있는 Sub-graph는 Deterministic Order에 의해서 Ordering 됨
 - 즉 Tree의 Child
- 합의 알고리즘은 Ordering된 Sub-graph들끼리의 순서를 정해주는 역할
 - Narwhal 에서의 Partial Order된 트랜잭션들을 Total Order해줌
- 기존의 리더 기반 합의 알고리즘은 모든 트랜잭션 Batch에 대한 Total Ordering
 - Narwhal과 결합하여 트랜잭션 전파와 정렬을 분리하면 모든 트랜잭션 Batch에 대한 합의X

Evaluation

- Mempool에서 Reliable Broadcast하는 프로토콜이 생겨나서 Latency는 증가하지만 TPS는 높음



B

▶ 예

Sui의 확장성 개선을 위한 Idea 정리

➤ Fast Path

- Object-Oriented Data Model 사용

- 모든 자산을 객체로 취급하고 각 객체를 독립적으로 관리하여 병렬처리 가능하게 함

➤ Narwhal-Bullshark

- DAG 기반 합의 프로토콜 도입
- 현재는 Mysticeti로 대체

Sui의 합의 알고리즘 구조

- Mempool 프로토콜과 합의 프로토콜의 구분
 - Mempool 프로토콜: 클라이언트가 생성한 트랜잭션을 수집하는 계층
 - 합의 프로토콜: 블록상 메타데이터의 순서를 결정하여 최종적으로 확정하는 계층
- 계층 분리 구조의 장점
 - Mempool 형성되는 과정에서 동시에 합의 프로토콜 수행 가능
 - 병렬처리를 통한 확장성 개선

Object Ownership

- 모든 객체에는 소유자 필드(Owner field)가 있음
 - 해당 객체를 트랜잭션에서 어떻게 사용할 수 있는지 결정
- 4가지 유형 중 하나의 소유권 가짐
 - **Address-owned 객체**
 - 특정 32byte address에 의해 소유됨
 - 소유자만 접근가능
 - **Immutable 객체**
 - 변경, 전송, 삭제 불가능
 - **Shared 객체**
 - 네트워크 모든 사용자 접근 가능하고 누구나 트랜잭션에서 사용 가능
 - **Wrapped 객체**
 - Struct 내에 다른 객체 포함해서 데이터 구성

객체 초기화 예시

- ShopOwnerCap 객체 생성하고 트랜잭션 실행한 계정에겐 전송 (Address-owned)
- DonutShop 공유 객체로 등록하여 모든 사용자가 접근하도록 설정

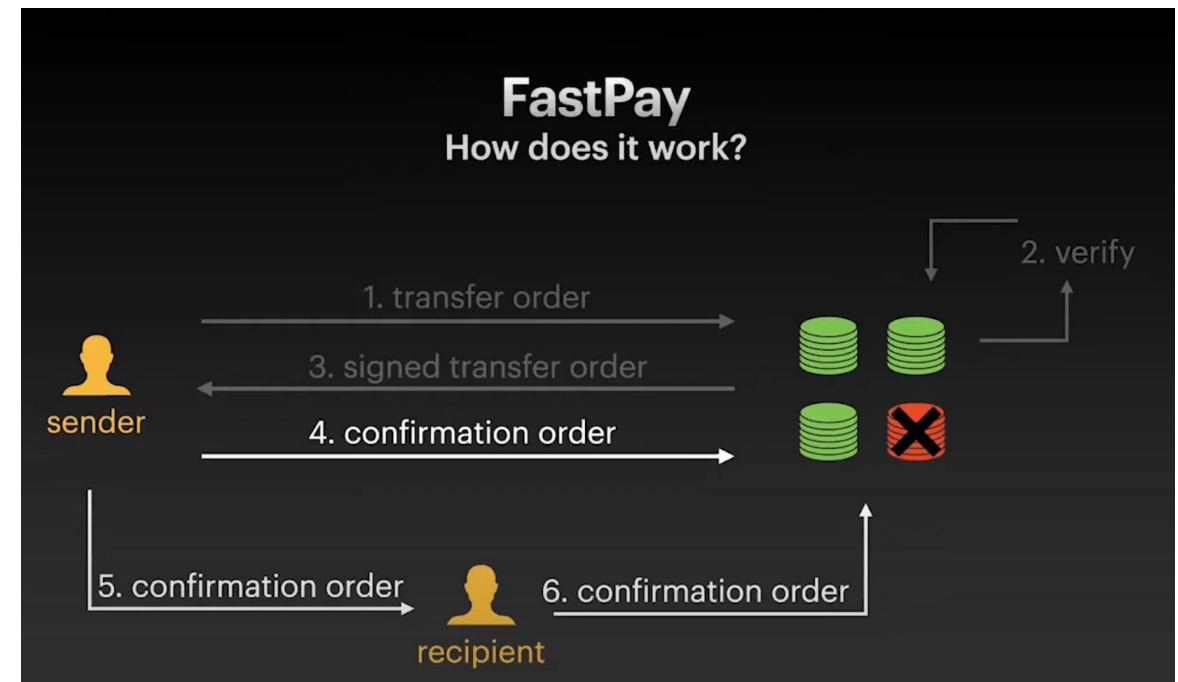
```
/// Init function is often ideal place for initializing
/// a shared object as it is called only once.
fun init(ctx: &mut TxContext) {
    transfer::transfer(ShopOwnerCap {
        id: object::new(ctx)
    }, tx_context::sender(ctx));

    // Share the object to make it accessible to everyone!
    transfer::share_object(DonutShop {
        id: object::new(ctx),
        price: 1000,
        balance: balance::zero()
    })
}
```

<Object-based Data Model 초기화 예시>

Consensus-less Blockchain

- Ordering 불필요한 트랜잭션에 대해서는 Byzantine Consistent Broadcast
- FastPay
 - UTXO Model
 - Single Writer Model
 - Owner만 수정 가능
 - Move에서의 Owned Object
 - Key concept
 - 사용자가 트랜잭션을 Authorities에게 전송하여 검증을 받음
 - $2f+1$ Threshold Signature 모으면 확정 메시지 보냄
 - 트랜잭션 확정되면 순서 없이 독립적으로 처리
 - ✓ 병렬처리 가능



<FastPay 작동 방식>

Consensus-less Blockchain 문제점

- Limited to asset transfers
 - Account Model에서의 자산은 대부분 shared object
 - 여러 당사자가 접근 및 소유하는 대상
 - Smart contract
 - Multi-owned objects
 - 당사자끼리의 Sequence Coordination이 어려움

Ordering을 위한 Consensus

- Shared Object에 대해서는 Consensus 필요
 - Sui-Lutris에서는 Narwhal-Bullshark DAG 기반 합의 프로토콜을 합의 엔진으로 사용
 - 현재 Sui에서는 Mysticeti를 합의 엔진으로 사용 중

공유 객체 사용 예시

- Sui 트랜잭션 단위는 Move 함수 호출 단위
 - buy_donut, collect_profit 함수는 Shared object 사용하니 합의 대상

```

/// Entry function available to everyone who owns a Coin.
public fun buy_donut(
  shop: &mut DonutShop, payment: &mut Coin<SUI>, ctx: &mut TxContext
) {
  assert!(coin::value(payment) >= shop.price, ENotEnough);

  // Take amount = `shop.price` from Coin<SUI>
  let coin_balance = coin::balance_mut(payment);
  let paid = balance::split(coin_balance, shop.price);

  // Put the coin to the Shop's balance
  balance::join(&mut shop.balance, paid);

  transfer::transfer(Donut {
    id: object::new(ctx)
  }, tx_context::sender(ctx))
}

```

새로운 객체 생성하고
구매자에게 전송

payment 코인에서 분리하여
DonutShop 계좌에 추가

```

/// Consume donut and get nothing...
public fun eat_donut(d: Donut) {
  let Donut { id } = d;
  object::delete(id);

  // Take coin from `DonutShop` and transfer it to tx sender.
  // Requires authorization with `ShopOwnerCap`.
  public fun collect_profits(
    _: &ShopOwnerCap, shop: &mut DonutShop, ctx: &mut TxContext
  ) {
    let amount = balance::value(&shop.balance);
    let profits = coin::take(&mut shop.balance, amount, ctx);

    transfer::public_transfer(profits, tx_context::sender(ctx))
  }
}

```

UTXO 모델처럼 코인
모아주는 과정이 필요

<Shared Object 트랜잭션 예시>

트랜잭션 전파와 정렬의 분리

- 일반적인 블록체인에서는 전파와 정렬이 하나의 과정에서 이루어짐
 - 각 노드가 트랜잭션 수신하고 이를 검증한 후, 자신의 Mempool에 저장
 - 이후 검증자에 의해 선택되어 블록에 포함되고 합의과정을 통해 블록체인에 추가
- 트랜잭션 데이터들이 합의 프로세스에 포함되지 않고 메타데이터 기반으로 합의 진행
 - 메타데이터: 블록 헤더와 해시들
- 데이터 전파 단계
 - Reliable Broadcast를 사용하여 트랜잭션 DAG 내에 삽입
 - DAG의 노드는 트랜잭션 포함하고 엣지는 이전 트랜잭션과의 관계 나타냄
- 합의 단계
 - DAG 내 메타데이터에 대해 합의하여 Global Order 결정

Proof of Availability

- 네트워크 내 특정데이터(e.g. transaction batch)가 실제로 사라지지 않고 안전하게 복제되어 언제든지 복구 가능함을 보장하는 메커니즘
- 문제상황
 - 분산환경에서 한 노드가 블록 제안했을 때 해당 데이터를 실제로 다수의 노드가 받았는지 확인 필요
- 핵심 아이디어
 - 데이터 네트워크에 전파한 뒤, 누가 데이터 보유하고 있는지에 대한 Proof 만들어 합의에 활용
 - $n-f$ 이상이 데이터 받음을 서명해주면 그 서명 모음 자체가 PoA가 됨
 - 나중에 원본 데이터 필요하면 Proof 근거로 해당 데이터 조각을 요청해 복원 가능
- 구현 예시
 - DAG기반 Mempool 시스템에서 각 Vertex에는 트랜잭션 원본이 아닌 Proof만 넣음
 - 서명, 해시, 인증 정보

B

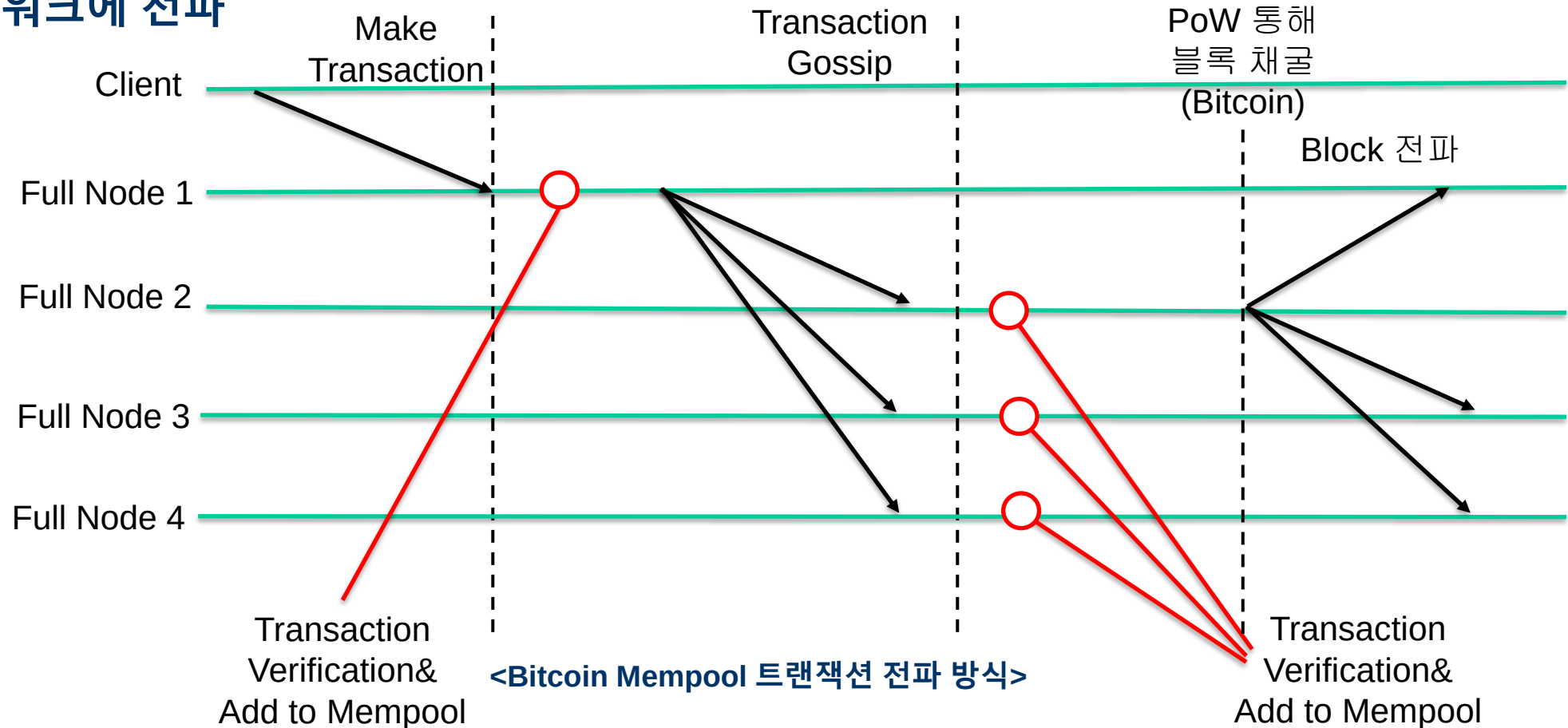
➤ 예

리더기반 알고리즘의 문제점

- 트랜잭션을 리더 혼자서 받기 때문에 확장성 개선 한계
- 리더 병목 현상
 - 모든 트랜잭션이 리더를 통해 처리되므로 리더 노드의 처리 능력이 시스템 최대 처리량 제한
 - 트랜잭션 처리 지연, 네트워크 혼잡, 리더 노드의 장애
- 트랜잭션을 모든 노드가 받는 방법은 어떨까

Bitcoin, LibraBFT의 트랜잭션 수집 방식

- 사용자가 네트워크에서 트랜잭션 전송하면 네트워크 모든 노드에 gossip
- 채굴자나 검증자는 Mempool에 있는 트랜잭션 중에서 우선순위에 따라 선택하여 블록 생성하고 네트워크에 전파



비트코인 방식의 Mempool 문제점

- 비트코인 Mempool의 문제점
 - 동일한 트랜잭션이 두 번 전파되는 이중 전파 문제 발생
 - Client의 트랜잭션 생성 후 전파
 - 검증자의 블록 생성 후 전파
- PoW가 아닌 합의 알고리즘을 사용한다면 Mempool 동기화도 문제

DAG properties

- 비동기 네트워크에서 노드마다 DAG가 일시적으로 달라도 결국 Total Ordering에 합의
- Validity
 - 정직한 노드가 로컬 DAG에서 정점 v 를 보유한다면, v 의 causal history 모두 가지고 있다
 - v 에서 도달 가능한 모든 정점
- Reliability
 - 정직한 노드가 어떤 노드가 어떤 라운드에서 만든 정점 가지고 있다면 결국 모든 정직한 노드들도 동일한 정점을 보게 된다
- Non-equivocation
 - 한 노드가 동일 라운드에 두개 이상의 정점을 만들어 제출할 수 없다
- Completeness
 - 두 정직한 노드에서 같은 정점은 그 정점이 참조하고 있는 히스토리도 같다

Transaction-Object DAG

- 트랜잭션은 객체를 입력으로 받아 읽고/수정하고/변형하여 새로운 객체 출력
- 트랜잭션과 객체 간의 관계를 DAG로 표현 가능
 - 각 객체는 마지막으로 이 객체 생성한 트랜잭션의 해시 저장
 - Node: 트랜잭션
 - Edge: txA->txB
 - A의 출력 객체가 B의 입력 객체로 사용될 경우
 - 객체 참조를 통해 어떤 객체 사용되었는지 나타냄
- Sui의 Global state는 모든 Live Object

Reliable Broadcast

- 비동기, 비잔틴 모델을 가정하더라도 다음의 속성 유지
 - Reliability: 모든 정직한 노드는 동일한 메시지를 수신해야 함
 - Atomicity: 모든 정직한 서버가 동일한 순서로 메시지 전달받아야 함
 - Secure Causality: 비정상적인 서버가 다른 정직한 서버보다 먼저 메시지 볼 수 없음